

COMPSCI 620 Reading Notes

Nikhil Anand

February 11, 2026

Contents

I	Reading Notes - February	2
1	[Required] Understanding Understanding Source Code with Functional Magnetic Resonance Imaging	3
1.1	Motivation	3
1.2	Methodology	4
1.3	Key Findings	5
1.4	Implications	6
2	[Required] An fMRI study of code review and expertise	8
2.1	Motivation	8
2.2	Methodology	9
2.3	Key Findings	11
2.4	Implications	11
3	[Skim] Why Is It Difficult for Programmers to Learn a New Programming Language	13
3.1	Summary	13
3.2	Methodology	13
3.3	Key Findings	14
3.4	Implications	14

Part I

Reading Notes - February

Chapter 1

[Required] Understanding Understanding Source Code with Functional Magnetic Resonance Imaging

The paper investigates whether functional magnetic resonance imaging (fMRI) can be used to directly measure what happens in the brain when programmers understand source code. The broader aim is to build a scientific, cognitive basis for program comprehension so that programming tools, languages, and education can be improved.

1.1 Motivation

The main motivation of the paper is that we still don't fully understand what happens cognitively when programmers read and understand code, even though software underpins modern society. Researchers and educators want to improve programming languages, tools, and training, but most existing studies rely on indirect measures—such as performance, surveys, or think-aloud protocols—which are noisy, subjective, and often inconclusive.

But, why do we need a better understanding of program comprehension? Understanding program comprehension is not limited to theory building, but can have real downstream effects in improving education, training, and the design and evaluation of tools and languages for programmers. If direct measures of cognitive effort and difficulty could be obtained and correlated with programming activity, then researchers could identify and quantify which types of activities, segments of code, or kinds of problem solving are troublesome or improved with the introduction of a new language or tool.

The authors argue that to design better tools, languages, and educational approaches, we need direct, objective measurements of the mental processes involved in program comprehension. If researchers could measure cognitive effort and difficulty while someone reads code, they could identify which code structures are hardest to understand, evaluate new tools or languages more rigorously, and better train programmers.

To address this gap, the paper proposes using functional magnetic resonance imaging (fMRI) to observe brain activity during code comprehension. fMRI is widely used in cognitive neuroscience to study complex mental processes, so the authors aim to test whether it is feasible and meaningful to apply it to programming. Their broader goal is to lay the groundwork for a scientific understanding of program comprehension that could eventually answer broader questions, such as:

- What cognitive processes are involved in understanding code?
- How do languages, tools, or training methods affect comprehension?
- Can we predict or improve programming ability?

1.2 Methodology

1.2.1 fMRI

The authors use functional magnetic resonance imaging (fMRI) in a controlled experiment to measure brain activity while participants understand source code. fMRI is a non-invasive brain-imaging technique that measures changes in blood oxygenation, the BOLD (blood oxygen level dependent) signal, to infer neural activity. When a brain region becomes active, it consumes more oxygen, and this change in oxygenated vs. deoxygenated blood can be detected by the scanner. From these signals, researchers can determine which brain areas are active during a task. Because decades of neuroscience research have mapped many brain regions (Brodmann areas) to cognitive functions like language, memory, and attention, observing activation patterns allows researchers to infer which mental processes are involved in program comprehension.

1.2.1.1 Why was fMRI chosen?

The authors chose fMRI because:

- It provides direct physiological measurements of cognitive activity, unlike surveys or performance metrics.
- It allows identification of specific brain regions involved in understanding code.
- It is well established in cognitive neuroscience for studying complex behaviors like language comprehension.
- It can help build a scientific foundation for understanding programming cognition.

1.2.1.2 Challenges

1. fMRI does not measure neural activity directly—it measures the BOLD (blood oxygen level dependent) response, which reflects changes in blood oxygenation when a brain region becomes active. This signal takes a few seconds to appear after a cognitive process begins and peaks around 5 seconds before returning to baseline about 12 seconds after the task ends. Because of this delay, tasks must be carefully timed and long enough (often 30–120 seconds) so that activation builds up and can be measured reliably. Short or poorly timed tasks would produce weak or noisy signals.
2. To obtain statistically reliable results, fMRI studies requires multiple repetitions of similar tasks, averaging brain activity across trials and a careful comparison between experimental and control conditions. This means experiments must use tasks that are consistent in difficulty and duration, which limits how realistic or complex the tasks can be.
3. fMRI is extremely sensitive to movement. Even small movements of the head can introduce artifacts in the signal.

1.2.2 Structure of the Experiment

The study followed standard fMRI experimental design principles:

- Participants lay still in a scanner with their head stabilized to avoid motion artifacts.
- Code snippets were shown on a small mirrored screen.
- Tasks were repeated multiple times to average brain activity.

Each trial included:

1. comprehension task (60s)
2. rest period
3. syntax task (30s)

4. rest period

Rest periods establish a baseline so researchers can compare task-related activation. Participants indicated mentally when they finished a task (using a button) to minimize movement, and correctness was verified afterward.

1.3 Key Findings

1.3.1 Program comprehension activates a specific network of brain regions

The central result is that understanding source code produced a clear and consistent activation pattern in five brain regions, all in the left hemisphere - Brodmann Area 6, 21, 40, 44 and 47. These regions were significantly more active during program comprehension than during syntax-error detection, meaning they are associated specifically with understanding code rather than just reading it or visually scanning it.

1.3.1.1 Working memory and attention play a central role

Two of the activated regions, especially BA 6 and BA 40, are strongly linked to - working memory, divided attention and problem solving. This suggests that when programmers understand code, they must actively:

- Hold variable values and intermediate results in memory
- Track control flow (loops, conditions)
- Coordinate multiple pieces of information simultaneously

In other words, code comprehension heavily relies on cognitive resources similar to those used in reasoning and problem-solving tasks.

1.3.1.2 Strong involvement of language-processing regions

Another major finding is that three activated areas (BA 21, 44, and 47) are typically associated with language processing. This indicates that understanding code engages brain systems used for:

- Processing words and symbols
- Combining them into meaningful structures
- Interpreting semantics

The authors conclude that program comprehension shares similarities with both natural-language and artificial-language processing. This provides empirical support for earlier claims (e.g., Dijkstra) that language skills are important for programming.

1.3.1.3 Evidence for a bottom-up comprehension process

Because the experiment used short code snippets with obfuscated identifiers, it emphasized bottom-up comprehension (understanding code step by step rather than relying on prior knowledge). The activation pattern suggests that programmers:

- Read symbols and words
- Combine them into statements
- Integrate statements into larger semantic chunks
- Keep intermediate values in working memory

This supports a cognitive model in which code comprehension involves incremental integration and memory tracking.

1.3.2 Left-hemisphere dominance

All major activations occurred in the left hemisphere, which is typically associated with analytical reasoning and language. This reinforces the idea that program comprehension relies on structured, language-like cognitive processes.

1.3.3 Feasibility of using fMRI in software-engineering research

Beyond cognitive findings, an important result is methodological: The study demonstrates that fMRI can successfully be used to study program comprehension and produce meaningful activation patterns. This opens the door for future research on programming cognition using neuroscience methods.

1.4 Implications

The results of the paper open several important directions for future research and practical applications.

1.4.1 Study different types of program comprehension

This paper focused mainly on bottom-up comprehension (reading unfamiliar code line by line). A natural next step is to study other forms of comprehension, especially:

- Top-down comprehension: understanding code using prior knowledge, domain familiarity, or meaningful variable names
- Realistic large programs: moving beyond short snippets to more complex codebases
- Use of identifiers and domain knowledge: testing whether meaningful variable names act as “beacons” that change brain activation

Future experiments could compare obfuscated vs. meaningful code to see how memory and recognition affect comprehension.

1.4.2 Study code writing, not just code reading

This study only examined understanding code. Another major step is to study code generation or editing, which may involve different cognitive processes such as creativity and synthesis. Possible experiments include

- Compare brain activity during coding vs. reading
- Examine whether writing code activates right-hemisphere or creative regions
- Study subvocal speech or internal narration while coding

This would help build a more complete model of programming cognition.

1.4.3 Compare novice and expert programmers

A key open question is how expert programmers differ from average programmers. Future work could:

- Compare activation patterns of novices vs. experts
- Study whether experts show more efficient or reduced brain activation
- Identify cognitive traits linked to programming skill

Such research might help predict programming aptitude or guide training methods.

1.4.4 Evaluate programming languages and tools

The methodology could be used to evaluate whether certain language features or tools make code easier to understand. Possible experiments:

- Compare object-oriented vs. functional code
- Test different naming conventions or syntax styles
- Evaluate IDE tools or visualization systems
- Measure cognitive load across language designs

fMRI could help determine whether a language feature actually reduces mental effort rather than relying on subjective reports.

1.4.5 Compare code comprehension with natural language

Because the study found strong language-related brain activation, future work could compare:

- Code comprehension vs. reading natural language passages
- Programming languages vs. artificial grammars
- Multilingual programmers vs. monolingual programmers

This could clarify whether programming is more like language comprehension or mathematical reasoning.

1.4.6 Overall implication

The most important implication is that this paper establishes a new research direction: using neuroscience tools to study programming. The next steps involve scaling up the realism of tasks, comparing different kinds of programmers and programming activities, and using brain-based measurements to inform education, language design, and tool development.

Chapter 2

[Required] An fMRI study of code review and expertise

This paper studies how the brain processes computer code compared to natural language using fMRI. The researchers asked 29 participants to perform three tasks: code comprehension, code review, and prose (English text) review, while their brain activity was recorded. *They found that programming languages and natural language are represented differently in the brain.* Using machine learning on brain activity data, they could classify which task a participant was performing with high accuracy. Interestingly, the same brain regions were involved in distinguishing code from prose across tasks. *The study also found that expertise matters: more experienced programmers showed less difference in brain activity between code and prose tasks.* This suggests that as programmers gain skill, their brains may process code more similarly to natural language.

2.1 Motivation

The central motivation of this paper is that software engineering heavily depends on human judgment, yet we know very little about how the brain actually processes programming tasks. Activities like code comprehension and code review are fundamental to software development, but research in this area has mostly relied on behavioral studies, surveys, performance metrics, or self-reports. These methods tell us what developers do, but not how their brains represent and process code. The authors argue that this is a major gap. Developers spend a large portion of their time understanding code, and companies like Google and Facebook require formal code review for new contributions. Code understanding is often ranked as more important than even functional correctness in real-world software contexts. Despite this practical importance, there is almost no neuroscientific understanding of how programming is represented in the brain.

Another key motivation is the relationship between programming languages and natural languages. Prior research has shown that software has statistical properties similar to natural language. This raises an important question: does the brain process code like it processes English? Or does programming rely on completely different neural systems? Understanding this distinction could influence how we teach programming, how we design tools, and how we conceptualize programming as a cognitive activity.

The paper is also motivated by expertise. There are well-documented productivity differences between novice and expert programmers, sometimes by an order of magnitude. Other fields like physics, music, or taxi driving have shown that expertise can physically alter brain organization. However, no prior work had examined whether programming expertise changes neural representations. The authors aim to explore whether more experienced programmers process code differently at a neural level.

Beyond curiosity, the authors outline several broader reasons for using medical imaging in software engineering:

- Self-reports are often unreliable; brain data could provide more objective insight.
- Understanding neural processing could improve pedagogy by clarifying whether programming is cognitively closer to language, math, or something else.

- It may help explain how expertise develops and why experts are more efficient.
- It could guide tool design by revealing how humans mentally evaluate code.

In short, the paper is motivated by a fundamental question: what is the neural basis of programming? By using fMRI, the authors aim to move beyond behavioral performance and directly examine how code and prose are represented in the brain, and how that representation changes with expertise.

2.1.1 Background

Most of the background is quoted from the paper.

2.1.1.1 Code Review

Static program analysis methods aim to find defects in software and often focus on discovering those defects very early in the codes lifecycle. Code review is one of the most commonly deployed forms of static analysis in use today; well-known companies such as Microsoft, Facebook, and Google employ code review on a regular basis. At its core, code review is the process of developers reviewing and evaluating source code. Typically, the reviewers are someone other than author of the code under inspection. Code review is often employed before newly-written code can be committed to a larger code base. Reviewers may be tasked to check for style and maintainability deficiencies as well as defects.

2.1.1.2 Code comprehension

Much research, both recent and established, has argued that reading and comprehending code play a large role in software maintenance. A well-known example is Knuth, who viewed this as essential to his notion of Literate Programming. He argued that a program should be viewed as *“a piece of literature, addressed to human beings”* and that a readable program is *“more robust, more portable, [and] more easily maintained.”* Knight and Myers argued that a source-level check for readability improves portability, maintainability and reusability and should thus be a first-class phase of software inspection.

2.1.1.3 Computer Science and Neuroscience

Siegmund et al. presented the first publication to use fMRI to study a computer science task: code comprehension. The focus of their work was more on the novelty of the approach and the evaluation of code comprehension and not on controlled experiments comparing code and prose reasoning.

Some researchers have begun to investigate the use of functional near-infrared spectroscopy (fNIRS) to measure programmer blood flow during code comprehension tasks. In smaller studies involving eleven or fewer participants, Nakagawa et al. found that programmers show higher cerebral blood flow when analyzing obfuscated code and Ikutani and Uwano found higher blood flow when analyzing code that required memorizing variables. fNIRS is a low-cost, non-invasive method to estimate regional brain activity, but it has relatively weak spatial resolution (only signals near the cortical surface can be measured); the information that can be gained from it is thus limited compared to fMRI. These studies provide additional evidence that medical imaging can be used to understand software engineering tasks, but neither of them consider code review or address expertise.

2.2 Methodology

2.2.1 fMRI

Functional magnetic resonance imaging (fMRI) is a non-invasive neuroimaging technique that measures brain activity by detecting changes in blood flow. When a specific brain region is more active, it consumes more oxygen, leading to increased blood flow to that area. fMRI detects these changes in blood oxygenation levels, allowing researchers to infer which regions of the brain are involved in specific cognitive tasks. In this study, fMRI was used to compare brain activity during code comprehension, code review, and prose review tasks.¹

¹The authors do point out that the use of fMRI in Software Engineering is still exploratory and that there are limitations to the technique

2.2.2 Experiment Design

2.2.2.1 Participants

Thirty-five students at the University of Virginia were recruited for this study. Solicitations were made via fliers in two engineering buildings and brief presentations in a third-year “Advanced Software Development Techniques” class and a fourth-year “Language Design & Implementation” class. Out of these 35 participants, 29 were included in the final analysis after excluding those with excessive head motion during scanning. All of the participants were,

- Right-handed
- Native English speakers
- Screened for programming experience and MRI safety

The final sample comprised of 18 males and 11 females. Additional objective and subjective measures of programming experience were collected, including years of programming experience, number of programming languages known and GPA.

2.2.2.2 Tasks

Participants performed three tasks while undergoing fMRI scanning:

- **Code Comprehension:** Participants read a short code snippet and answered a true/false question about its behavior.
- **Code Review:** The participants were shown a Github style pull request, code differences and a comment from a reviewer. They were asked to evaluate the comment and decide to accept or reject the pull request.
- **Prose Review:** Participants were shown a short English passage and suggested edits. They were asked to evaluate the edits and decide to accept or reject them.

2.2.3 Preprocessing

Before analysis, the raw brain data underwent standard preprocessing for motion correction (head movement adjustment), alignment to a standard brain template (MNI space), noise correction, and no spatial smoothing was applied. This ensured that the data was clean and comparable across participants, while preserving the fine-grained spatial information necessary for multivariate pattern analysis (MVPA).

2.2.4 Processing

The extremely large dimension of fMRI data is a major obstacle for machine learning — we commonly have tens of thousands of voxels² but only a few dozen training examples. This can be solved using a simple linear map (a kernel function) that reduces the dimensionality of the feature space. The training inputs were given as a set of vectors $\{x_n\}_{n=1}^N$, with corresponding labels $\{y_n\}_{n=1}^N$ where N is the number of beta images for each participant across both classes. To learn a mapping, the authors employed a cumulative Gaussian, and specified posterior conditional over f using the Bayes rule

$$p(f|\mathcal{D}, \theta) = \frac{\mathcal{N}(f | 0, K)}{p(\mathcal{D} | \theta)} \prod_{n=1}^N \phi(y_n f_n)$$

where $\mathcal{N}(f | 0, K)$ is a zero-mean prior, ϕ is a factorization of the likelihood of training samples.

²A voxel is the smallest unit of an fMRI image, analogous to a pixel in a regular image.

2.3 Key Findings

2.3.1 Research Question 1

RQ1: Can we classify which task a participant is performing based solely on patterns of their brain activity?

Code Comprehension vs Prose tasks were highly distinguishable, with a 79% balanced accuracy ($p < 0.001$) and Code Review vs Prose tasks were also highly distinguishable, with a 71% balanced accuracy ($p < 0.001$). This strongly suggests that the brain forms distinct neural representations for programming tasks versus natural language tasks. Code is not processed identically to English prose and even though Code Review and Code Comprehension share similarities, but still differ at the neural level.

2.3.2 Research Question 2

RQ2: Can we relate these task differences to specific brain regions?

The same set of brain regions distinguished code review and comprehension from prose. The regional importance maps were almost perfectly correlated ($r = 0.99, p < 0.001$). Highly involved regions included the prefrontal cortex (executive control, reasoning, decision-making) and the language-related areas near Wernicke’s area.

This means that code and prose differ consistently in how they activate higher-order cognitive regions. Programming is not processed purely as math or purely as language. It heavily recruits executive control regions (working memory, abstract reasoning). But language-related areas are also involved. This finding suggests programming is a hybrid cognitive activity, involving language systems and executive reasoning systems. And importantly, the same neural system distinguishes code from prose across different kinds of code tasks.

2.3.3 Research Question 3

RQ3: Does expertise influence neural representation of code vs prose?

This is one of the most interesting parts of the paper. The authors measured expertise using CS GPA (corrected for number of credits), then correlated it with classifier accuracy. There was a negative correlation between expertise and classification accuracy ($r = -0.44, p = 0.016$) for code comprehension vs prose.

This means that as programmers become more skilled, their brain activity for code and prose becomes less distinguishable. Experts process code in a way that is more similar to natural language. With experience, code may become more “language-like” in neural representation. Expertise leads to more integrated cognitive processing. This aligns with findings from other domains (e.g., physics, music, taxi driving) where expertise reorganizes neural representations.

2.4 Implications

Programming Has a Distinct Neural Basis The finding that code and prose can be reliably classified from brain activity implies that programming is not simply “reading English with symbols.” It recruits distinct neural representations. This challenges the common assumption that programming is purely a linguistic activity. Instead, it appears to involve a unique cognitive configuration that blends language and executive reasoning systems. This has foundational implications: programming should be understood as its own cognitive domain, rather than being reduced to either language or mathematics. It justifies studying programming cognition as a standalone scientific field.

Programming Is a Hybrid Cognitive Activity Although code and prose are distinct, they activate overlapping language-related regions along with executive control regions (e.g., prefrontal cortex). This suggests programming is cognitively hybrid — it involves structured language processing but also heavy working memory, abstraction, planning, and decision-making. The implication here is that programming education and research should not frame coding as purely syntactic or linguistic. Instead, it requires

integrated training in reasoning, control processes, and structured abstraction. This supports the idea that difficulty in programming may arise not from syntax learning, but from cognitive load and executive demand.

Expertise Changes Neural Representation One of the most profound implications is that increased programming expertise reduces the neural distinction between code and prose. This suggests that with experience, code becomes cognitively more “natural.” Experts may internalize programming structures similarly to how fluent readers process language. This supports theories of neural plasticity — expertise reorganizes how information is represented in the brain. It implies that the dramatic productivity gap between novice and expert programmers may not just be skill-based, but neurologically grounded. Programming expertise appears to alter mental representation itself.

Rethinking Code as “Natural Language” Previous research showed software shares statistical properties with natural language. This paper adds neural evidence to the discussion. While code is statistically language-like, it is neurally distinct — especially for novices. However, expertise moves it closer to language representation. This nuanced finding suggests that programming may be “learned language,” rather than inherently language-like. It strengthens interdisciplinary connections between cognitive science, linguistics, and software engineering.

Chapter 3

[Skim] Why Is It Difficult for Programmers to Learn a New Programming Language

3.1 Summary

This paper explores why experienced programmers still struggle when learning a new programming language, even though they already know how to program. The core idea is something called **cross-language interference**. When developers learn a new language, they naturally try to reuse knowledge from languages they already know. Sometimes this helps — but very often, it leads to incorrect assumptions. These wrong assumptions cause confusion and mistakes.

3.2 Methodology

3.2.1 Phase 1: Stack Overflow Study

In Phase I, the researchers conducted an empirical study using Stack Overflow posts to investigate whether cross-language interference actually occurs in practice. They collected 450 posts across 18 programming language pairs, focusing on situations where programmers were transitioning from one language to another. To identify relevant posts, they used filtering criteria that captured questions referencing both a source language and a target language. The goal was to detect cases where developers were implicitly or explicitly bringing assumptions from a previously known language into a new one.

The researchers manually inspected and categorized the posts into two types: correct assumptions (facilitation) and incorrect assumptions (interference). A post was labeled as interference if the programmer made an incorrect assumption about the new language based on prior experience. To ensure reliability, two researchers independently coded the data and achieved high inter-rater agreement (Cohen's $K = 0.89$), indicating strong consistency in classification. Their analysis revealed that 61% of the posts contained incorrect assumptions caused by cross-language interference, providing large-scale evidence that prior programming knowledge can negatively impact learning a new language.

3.2.2 Phase 2: Interviews with Professional Programmers

In Phase II, the researchers conducted semistructured interviews with 16 professional programmers to gain deeper insight into how experienced developers learn new languages and how interference manifests in real-world learning. The participants had between 5 and 31 years of programming experience and had recently transitioned to a new language. These interviews lasted approximately 60 minutes and explored learning strategies, challenges faced, surprising differences, and the role of prior knowledge during the transition process.

The researchers used inductive thematic analysis to analyze the interview transcripts. They coded recurring patterns and grouped them into broader themes that described how programmers approach

language learning and where difficulties arise. This qualitative phase revealed that experienced developers often rely on opportunistic, self-directed learning strategies, frequently trying to relate new languages to ones they already know. However, when fundamental differences exist—such as paradigm shifts, syntax differences, or tooling ecosystem changes—these prior mental models can become sources of confusion. This phase helped explain the cognitive and practical mechanisms behind the interference observed in Phase I, complementing the large-scale data with rich experiential insights.

3.3 Key Findings

Cross-language interference is common and measurable The study found strong evidence that prior programming knowledge often interferes with learning a new language. In their Stack Overflow analysis, 61% of the 450 posts examined contained incorrect assumptions caused by transferring knowledge from a previously known language. This shows that interference is not rare or anecdotal — it is widespread across many language pairs. Importantly, interference appeared across diverse transitions, not just between very different languages.

Prior knowledge is a double-edged sword Previous programming experience does not uniformly help. The authors found both:

- **Facilitation:** When knowledge transfers correctly and helps learning.
- **Interference:** When prior assumptions lead to misunderstanding.

Whether knowledge helps or harms depends heavily on how similar the two languages are — syntactically, conceptually, and paradigmatically. When languages share surface similarities but differ in deeper semantics, programmers are especially prone to errors.

Experienced programmers rely on opportunistic learning From interviews, the researchers found that professionals rarely learn new languages systematically (like reading a full book front to back). Instead, they use just-in-time learning strategies — Googling errors, using cheat sheets, browsing documentation, and asking colleagues. They often try to quickly relate the new language to one they already know. While efficient, this strategy increases the risk of shallow understanding and incorrect mental mappings.

Paradigm shifts create major difficulty Transitions involving paradigm changes (e.g., object-oriented → functional, or manual memory → ownership models) were particularly difficult. Developers had to actively suppress old habits and rethink how problems are structured. This mental restructuring — not syntax — was often the biggest challenge.

Tooling and ecosystem differences add friction Learning a new language is not just about syntax and semantics — it also involves adapting to new IDEs, package managers, debugging tools, and workflows. The interviews revealed that ecosystem transitions can be just as cognitively demanding as language-level differences.

Lack of explicit transition support The authors observed that documentation and learning resources are usually designed for beginners, not experienced programmers switching languages. There is little structured support for cross-language transitions, even though interference is common.

3.4 Implications

Documentation should explicitly support language transitions The findings suggest that documentation and learning resources should not assume learners are complete beginners. Many programmers already know other languages and try to map new concepts onto prior knowledge. Instead of generic tutorials, resources could include transition-focused guides such as “If you’re coming from Java...” or “For Python developers...”. Making similarities and differences explicit can reduce incorrect assumptions and help programmers build accurate mental models faster.

Tools can reduce interference through smarter feedback The study highlights opportunities for IDEs and compilers to provide context-aware error messages that anticipate common misconceptions. For example, if a developer coming from Python makes a Java-style mistake, the system could detect that pattern and explain the difference explicitly. This kind of adaptive tooling could help programmers correct faulty assumptions before they solidify into confusion.

Language designers should anticipate prior mental models When designing new languages, designers should consider how programmers from popular ecosystems will interpret features. Some features may look familiar but behave differently, increasing the risk of interference. Being mindful of how existing mental models transfer — or clash — can reduce friction during adoption. In other words, usability across language boundaries matters.

Learning a language means entering an ecosystem The study shows that difficulty is not limited to syntax or semantics. Tooling, IDEs, debugging workflows, package managers, and build systems also create friction. Therefore, improving onboarding into an ecosystem (not just the language itself) is crucial. Streamlined setup tools, unified build environments, and consistent tooling conventions can ease transitions.

Programming knowledge does not fully generalize A broader implication is theoretical: programming expertise does not automatically transfer smoothly across languages. Mental models are often language-specific. This challenges the assumption that “once you know one language, the rest are easy.” For educators and researchers, this means we need better frameworks for understanding knowledge transfer and interference in programming.